

AI-NATIVE

Solution Architecture Conceptual Reference

Agents • MCP • RAG • Middleware • Orchestration • Tokens • Embeddings
LangChain • LangGraph • LangSmith • FastAPI • Python • Enterprise AI
Authoritative Source Edition — official / standards / research links verified 2 Oct 2026

Purpose

A reusable reference for understanding how modern AI application components fit together. The emphasis is conceptual architecture: what each component does, why it exists, when to use it, and how it maps to a real enterprise system such as AIRA.

Audience: experienced software / solution architect moving into AI-native architecture

0. How to use this document

This is not meant to replace official product documentation or teach every API. It is your architecture “map”. Use it whenever terminology becomes confusing, especially when several tools appear to solve similar problems.

- First learn the mental model, then learn a framework. Do not memorize framework APIs before you understand the underlying system behavior.
- For every new technology, ask four questions: What problem does it solve? What does it replace or complement? Where does it run? What can go wrong?
- Keep the AIRA example in your head. Nearly every concept in this document is mapped to AIRA so the theory becomes design vocabulary.
- Treat “Agent”, “Workflow”, “Orchestrator”, “Tool”, “MCP”, and “RAG” as different concepts. They can work together, but none is a synonym for the others.

Architect rule

Frameworks change. Architectural responsibilities persist. If LangChain, LangGraph or another

framework changes, your understanding of state, retrieval, tools, security, orchestration, evaluation and failure handling still transfers.

1. Reference map

Section	Topic
2	AI / LLM foundation
3	Agent
4	Tools and function calling
5	Orchestrator
6	Workflow vs agent
7	Middleware
8	MCP
9	RAG
10	Tokenization
11	Vectorization / embeddings
12	Vector database and retrieval
13	Context window and context engineering
14	Prompt engineering
15	Memory
16	LangChain
17	LangGraph
18	LangSmith
19	FastAPI
20	Minimum Python for AI
21	Supporting skills you actually need
22	Enterprise AI security and reliability
23	AIRA: complete conceptual architecture
24	AIRA 403 troubleshooting example
25	Choosing the right pattern
26	Study path and practice labs

Section	Topic
27	One-page cheat sheet
28	Source library

2. AI / LLM foundation

At the center of an AI application is a model. In a production system, the model is rarely the whole system. The application surrounds the model with instructions, context, tools, data retrieval, memory, orchestration, security and observability.

Layer	Question it answers	Typical component
User experience	How does a person or application interact with AI?	Web UI, chat UI, POS/ERP screen, API client
API / entry point	How does the request enter the AI system?	FastAPI, ASP.NET Core API, API gateway
Orchestration	What steps should run, in what order?	Application code, LangGraph, Agent Framework
Agent / decision layer	Which action or tool should be used next?	LLM + tool-calling agent
Knowledge	What private / current information is needed?	RAG, search index, SQL, APIs
Tool / action layer	What can the AI actually do?	Functions, APIs, MCP servers
Model layer	How does the system interpret / generate?	LLM, embedding model, reranker
Control layer	What must be blocked, retried, approved or logged?	Middleware, guardrails, policy, auth
Observability / evaluation	Did it work and is it getting better?	LangSmith, traces, evals, metrics

2.1 What is an LLM?

A Large Language Model (LLM) is a model trained to predict and generate sequences of tokens. For application architecture, think of the LLM as a reasoning-and-language engine rather than a database, transaction system or source of guaranteed truth.

Mental model

LLM = powerful probabilistic component. Your application = the system that provides context, tools, permissions, memory, validation and business rules around it.

2.2 What an LLM does not automatically provide

- Reliable access to your private ERP/POS data: add retrieval or tools.

- Permission to perform business actions: enforce application-side authorization.
- Deterministic workflow control: use program code or an orchestration runtime.
- Guaranteed factuality: ground responses in source data and evaluate outputs.
- Infinite context: token/context limits still matter, so retrieval, summarization and context management are architectural concerns.

2.3 The smallest useful AI application

User question
 -> Application
 -> LLM
 -> Text response

This is useful for general explanations or drafting, but it becomes insufficient when the task depends on proprietary data, external systems, multi-step actions, or strong reliability requirements.

3. Agent

Conceptually, an agent is an AI system in which the model participates in deciding what to do next. The modern pattern is a loop: receive goal -> inspect available tools/context -> choose an action -> observe the result -> decide whether another action is needed -> finish or ask for human input. LangChain currently describes its agent as a model calling tools in a loop until the task is complete. Anthropic similarly distinguishes fixed workflows from systems where the model dynamically directs its process and tool use.

Component	Purpose	AIRA example
Goal / instruction	Defines the job and behavioral boundaries.	Investigate an authentication failure and explain likely cause.
Model	Decides or generates the next response/action.	Selects whether to search KB, inspect logs or query customer data.
Tools	Provide data or actions beyond the model.	Jira search, log query, customer lookup, deployment status.
State / context	Carries the current investigation.	Tenant, request ID, prior findings, tool results.
Guardrails	Restrict unsafe or invalid behavior.	Read-only investigation until human approval is granted.
Stop conditions	Prevent endless or costly loops.	Max steps, confidence threshold, timeout, human handoff.

3.1 Agent vs chatbot

Chatbot	Agent
Mostly answer generation	Answer generation + decision + tool use
Usually fixed request/response	Iterative loop

Chatbot	Agent
Limited external action	Can query systems and take approved actions
Simple state	Execution state and possibly memory
Low orchestration complexity	Needs control, observability and safety boundaries

3.2 The agent loop

```

while not finished:
    context = build_context(state)
    decision = model.decide(context, tools)

    if decision.type == "tool_call":
        result = execute_tool(decision)
        state.add(result)
    else:
        return decision.answer

```

Architectural insight

The agent is not “the whole application”. It is the decision-making loop inside the application. API, identity, data, tools, policies, orchestration and observability remain separate architectural responsibilities.

4. Tools and function calling

A tool is a callable capability exposed to the model. It may read data, calculate something, search, call a business API, or perform a write action. Function calling is the mechanism by which the model requests your application to execute a defined tool and return the result to the model.

```

Model sees:
get_customer(customer_id: string)
get_api_logs(request_id: string)
create_jira_issue(title: string, description: string)

```

Model -> tool call -> application executes -> tool result -> model

- Tools should have clear names, precise descriptions, typed inputs and predictable outputs.
- Perform authorization and input validation in application code. Never trust the model merely because it produced a valid tool call.
- Keep the tool surface focused. Too many tools increase selection complexity and token usage.
- Make read operations easy and low-risk; require stronger controls, approval and auditing for writes or irreversible actions.

5. Orchestrator

An orchestrator coordinates work. It decides which components participate, in what order, with which state and under which failure rules. The important point is that orchestration is broader than an agent loop.

Orchestrator responsibility	Example
Route	Send billing questions to billing workflow; technical issues to AIRA investigation.
Sequence	Authenticate -> retrieve tenant -> classify -> retrieve -> analyze -> respond.
Parallelize	Fetch Jira, logs and service health concurrently when independent.
Retry / timeout	Retry a transient search call; terminate after deadline.
State management	Carry tenant ID, request ID, evidence and approvals across steps.
Human handoff	Pause before executing a risky action.
Audit	Record decisions, tool calls, approvals and outcomes.

5.1 Orchestrator vs agent

Simple rule

Orchestrator controls the overall process. Agent controls a model-driven decision loop inside that process. An orchestrator may contain zero, one or many agents.

6. Workflow vs agent vs orchestrator

These terms are often mixed together. Keep them separate by asking where control flow comes from.

Pattern	Who determines next step?	Predictability	Typical use
Deterministic workflow	Code / rules	High	Order processing, approvals, scheduled jobs
Agent	Model within guardrails	Lower / adaptive	Open-ended investigation, research, support diagnosis
Orchestrated hybrid	Code + model at selected decision points	Medium / controlled	Enterprise AI with reliable process and flexible reasoning

Architect rule

Use the simplest control model that satisfies the business requirement. Do not turn a predictable three-step process into an autonomous agent just because an agent framework is available.

7. Middleware

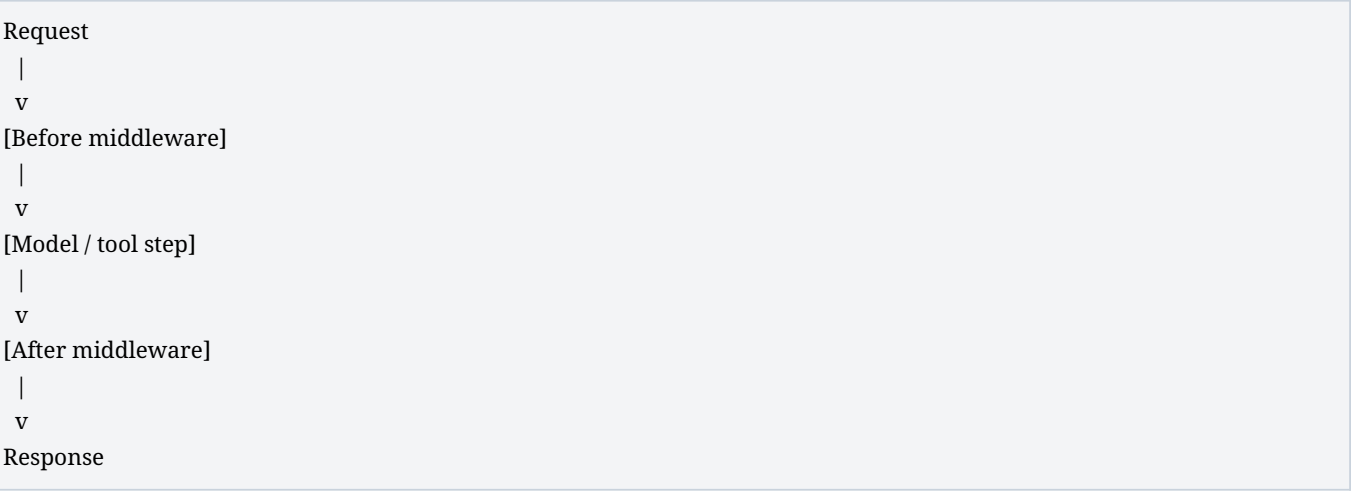
Middleware is code that intercepts execution to apply cross-cutting behavior without rewriting the core business logic. In web applications this can mean HTTP request/response processing. In LangChain agent systems, middleware can hook into agent execution to inspect, transform, retry, guard, summarize, route or terminate execution.

7.1 Two meanings you should know

Type	Where it sits	Example
API middleware	Between HTTP request and endpoint	Auth, correlation ID, rate limit, request logging.
Agent middleware	Inside / around agent execution	PII detection, dynamic prompts, retries, tool filtering, human-in-the-loop.

7.2 LangChain middleware

The current LangChain docs describe middleware as a way to control agent execution at every step. It can be used for logging, prompt/tool/output transformation, retries/fallbacks, rate limits, guardrails and PII detection. The middleware hooks execute inside the LangGraph runtime used by LangChain agents.



8. MCP - Model Context Protocol

MCP is a standard for connecting AI applications to external systems such as data sources, tools and workflows. The easiest analogy is “USB-C for AI”: the protocol standardizes how an AI application can discover and interact with capabilities exposed by an MCP server.

MCP concept	Think of it as	AIRA example
MCP host / AI application	The application using AI	AIRA agent runtime
MCP client	The protocol-side connector	AIRA connector to Jira MCP server
MCP server	Capability provider / adapter	Server exposing Jira, GitHub, Confluence or ERP tools
Tool	Callable operation	search_jira, get_issue, get_customer
Resource	Addressable data/context	Document, schema, record, file-like resource

8.1 MCP is not...

Not this	What it actually is
Not an agent	A connectivity protocol.
Not RAG	RAG retrieves knowledge; MCP standardizes access to external capabilities.
Not a vector database	MCP can expose a vector search tool; it is not the storage engine.
Not an orchestrator	It exposes capabilities; your application still controls orchestration.
Not a model	It does not generate reasoning or text.

8.2 Where MCP is valuable

- Standardizing integrations across many AI clients or agent frameworks.
- Separating enterprise system adapters from the AI orchestration code.
- Reusing connectors for Jira, GitHub, Confluence, databases or internal platforms.
- Establishing consistent discovery and invocation patterns for external capabilities.

9. RAG - Retrieval-Augmented Generation

RAG is a pattern for grounding an LLM response in information retrieved from an external knowledge source. It is particularly useful for private, organization-specific or frequently changing information that should not be assumed to be in the model's training data.

```

Documents -> clean -> chunk -> embed -> index
      |
User question -> retrieve relevant chunks --+
      |
      build context
      |
      LLM
      |
      grounded answer

```

9.1 Standard RAG

A standard RAG pipeline follows a relatively fixed sequence: receive question -> search -> collect top results -> assemble context -> call the model -> answer. Microsoft's current Azure Architecture Center guidance describes this fixed-pipeline pattern and contrasts it with agentic RAG, where retrieval itself becomes a tool an agent may invoke dynamically.

9.2 Agentic RAG

Agentic RAG is useful when retrieval is not a single predictable search. The agent may decide which source to query, whether another query is needed, how to decompose the question, and when enough evidence has been collected.

Question	Standard RAG	Agentic RAG
One well-defined knowledge search?	Good fit	Usually unnecessary
Multiple sources with dependencies?	Can become awkward	Good fit
Need dynamic query planning?	Limited	Natural fit
Need actions as well as retrieval?	Add tools around pipeline	Agent can combine retrieval and tools
Latency/cost sensitivity?	Usually simpler	More controls required

9.3 RAG pipeline concepts

Stage	Purpose	Architect questions
Ingestion	Bring documents/data into the knowledge pipeline.	What sources? What update frequency?
Cleaning	Normalize noisy source content.	What formatting / metadata should be retained?
Chunking	Split content into retrieval units.	What size and semantic boundaries work for the domain?
Embedding	Convert chunks to semantic vectors.	Which embedding model and dimension?
Index	Store searchable representations.	Vector, keyword, hybrid, filters?
Retrieval	Find likely relevant chunks.	Top-k? metadata filter? reranking?
Augmentation	Put retrieved evidence into model context.	What context format? citation metadata?
Generation	Produce grounded answer.	What instructions, tone, refusal behavior?
Evaluation	Measure retrieval and answer quality.	Which representative questions and acceptance criteria?

9.4 AIRA RAG example

Suppose the knowledge base contains POS authentication troubleshooting notes, IIS configuration guides, API gateway rules, known Jira incidents and deployment runbooks. AIRA can retrieve the most relevant chunks before asking the model to form its diagnosis.

Key distinction

RAG answers: “What relevant information can I retrieve?” An agent answers: “What should I do next, including whether I should retrieve something?”

10. Tokenization

Models process text as tokens rather than raw words. A token can be a whole word, part of a word, punctuation or another frequently occurring sequence. Token counts influence context capacity, latency and usage cost. OpenAI’s current concepts documentation also notes that context limits apply to the combined input and output for text generation models.

"authentication error"

|

v

[tokens / pieces]

|

v

model processing

10.1 Why architects care

- Prompt size: system instructions + conversation + tool schemas + retrieved chunks all consume context.
- Cost: larger inputs and outputs can increase usage cost depending on the model/provider.
- Latency: more input can increase processing time.
- RAG quality: too many retrieved chunks can bury the useful evidence inside irrelevant context.
- Agent tool surface: tool descriptions themselves can contribute to context usage.

10.2 Practical token control

- Retrieve fewer but better chunks; use metadata filters and reranking when needed.
- Summarize or compress long conversation history.
- Keep tool descriptions precise rather than verbose.
- Measure real token usage instead of relying on rough word-count assumptions.

11. Vectorization / embeddings

Vectorization converts content into a numeric representation that captures useful semantic relationships. An embedding is a vector representation of data such that semantically similar content tends to be

represented by nearby vectors. This enables semantic search, clustering, recommendations and related tasks.

Concept	Example
Text	“POS terminal returns 403 after token expiry.”
Embedding	[0.013, -0.271, ...] across many dimensions
Similarity	Compare question vector to stored document vectors.
Nearest neighbors	Return the chunks whose vectors are most similar.

11.1 Tokenization vs embedding

Tokenization	Embedding
Breaks text into model-processing pieces	Maps content into a semantic numeric vector
Primarily enables model input processing	Primarily enables semantic retrieval / similarity
Example: “auth” + “entication”	Example: vector for authentication troubleshooting meaning
Important for context size	Important for semantic search quality

Architect analogy

Tokenization is how the language model “cuts the sentence into pieces.” Embedding is how your retrieval system “places the meaning on a searchable map.”

12. Vector database and retrieval

A vector database or vector-capable search engine stores embeddings and allows efficient similarity search. In production RAG, you usually store both the vector and metadata such as tenant, document ID, source, timestamp, security classification and URL/file reference.

12.1 Semantic vs keyword vs hybrid

Search type	Strength	Weakness	Use
Keyword	Exact terms, codes, IDs, names	Weak on paraphrases	SKU, error code, API name, Jira key
Vector / semantic	Meaning and paraphrases	May miss exact codes / lexical constraints	“token expired” vs “authentication session timed out”
Hybrid	Combines lexical + semantic signals	More tuning	Enterprise knowledge bases with technical terminology

12.2 Metadata filtering is architecture, not decoration

In a multi-tenant system, the retrieval query must enforce tenant boundaries. AIRA should never rely on the LLM to remember that tenant A must not see tenant B's documents. The retrieval layer should filter by tenant and other authorization dimensions before the chunks reach the model.

13. Context window and context engineering

Context is everything the model is given for the current decision: system instructions, user input, conversation history, retrieved evidence, tool definitions, tool results and sometimes structured state. Context engineering is the architectural discipline of deciding what information reaches the model, when, in which form and with what priority.

Context source	Typical value	Risk if uncontrolled
System instructions	Behavior and policy	Conflicting / bloated instructions
User request	Immediate goal	Ambiguous intent
Conversation state	Continuity	Stale history / token growth
RAG evidence	Grounding	Irrelevant or conflicting chunks
Tool schemas	Action capability	Too many tools / token overhead
Tool results	Current system facts	Huge payloads / sensitive data

Rule of thumb

Do not ask “How much context can the model hold?” first. Ask “What is the minimum high-value context required to make this decision safely and correctly?”

14. Prompt engineering

Prompt engineering is the design of instructions and examples that guide model behavior. In an enterprise system, prompts are part of the application contract: they should define the task, constraints, grounding behavior, output format and edge-case handling.

14.1 A practical enterprise prompt structure

ROLE

You are an ERP/POS technical support investigator.

TASK

Determine the likely cause of the user's reported failure.

GROUNDING RULE

Use retrieved evidence and tool results as primary evidence.

If evidence is insufficient, say so.

CONSTRAINTS

Do not invent logs, tickets, IDs or configuration values.

Do not execute write actions without approval.

OUTPUT

Return: summary, evidence, likely cause, next checks, confidence.

14.2 Prompt is not security

A prompt saying “never expose confidential data” is not a substitute for authorization, filtering, secret management, network controls and application-side validation. Prompts influence model behavior; they do not replace deterministic security controls.

15. Memory

Memory means information retained across interactions or execution boundaries. Separate short-term execution state from long-term business memory. Many AI systems need only carefully designed state rather than a broad “memory” feature.

Type	What it stores	AIRA example
Execution state	Current task variables	Tenant, request ID, findings, steps completed
Conversation history	Recent messages	Previous turns in one support session
Long-term user preference	Stable user-specific information	Preferred explanation format
Business memory	Persistent domain facts	Previous confirmed root-cause resolution
Knowledge base	Shared reference information	Runbooks, manuals, KB articles

Important

RAG is not the same thing as memory. RAG retrieves external knowledge for grounding. Memory preserves state or prior interaction information. They can be combined but solve different problems.

16. LangChain

LangChain is an application framework that provides reusable building blocks for LLM applications and agents: model interfaces, tools, structured output, retrieval-related components, prompts, middleware and agent abstractions. The current LangChain documentation positions agents as model + harness, with tools and middleware shaping execution.

Need	LangChain role
Call a model	Standard model abstraction / provider integration
Expose a Python function as a tool	Tool abstraction / decorators
Build RAG	Retrievers and integrations
Create an agent	Higher-level agent construction
Control the agent loop	Middleware / integration with LangGraph
Integrate MCP	MCP support for connecting external capabilities

16.1 What LangChain is NOT

- Not the LLM.
- Not MCP.
- Not the vector database.
- Not your business process.
- Not your security boundary.
- Not the only way to build an AI system.

16.2 Architect mindset

Learn LangChain because it is a useful ecosystem abstraction and implementation toolkit. Do not become dependent on remembering every helper method. Be able to drop one layer down and explain what the framework is doing underneath: model request, tool schema, retrieval, state, retries and control flow.

17. LangGraph

LangGraph is a lower-level orchestration framework and runtime for long-running, stateful agent workflows. The current documentation emphasizes fine-grained control, durable execution, streaming, human-in-the-loop patterns and the ability to mix deterministic code with LLM-driven steps.

```

State
|
v
[Classify]
|
+--> [RAG search] ----+
|           |
+--> [Check logs] ----+--> [Analyze] --> [Human approval?] --> [Answer]
```

17.1 LangChain vs LangGraph

LangChain	LangGraph
Higher-level application / agent building blocks	Lower-level workflow and agent orchestration
Good starting point for common agents	Good when you need explicit state and control flow

LangChain	LangGraph
Focus on components and agent harness	Focus on graph topology, state and runtime behavior
Often used with LangGraph under the hood for agents	Can be used without LangChain

Simple mental model

LangChain = “what AI building blocks do I need?” LangGraph = “how does work move through states, branches, loops and approvals?”

18. LangSmith

LangSmith is an observability and evaluation platform for LLM applications. The current documentation describes traces as records of what agents did in production and provides tooling for debugging, monitoring quality, dashboards, alerts, feedback and evaluation datasets.

18.1 What a trace should let you answer

- What did the user ask?
- Which model version was used?
- Which prompt / middleware path ran?
- Which tools were called?
- What did retrieval return?
- How many tokens / how much latency occurred?
- Where did the run fail?
- Did the final answer meet the expected quality criteria?

18.2 LangSmith is more than logging

Architecture role

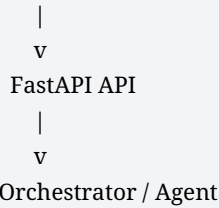
Logging tells you that something happened. Tracing shows the chain of reasoning-related execution steps. Evaluation tells you whether the result was good enough for your business requirement.

19. FastAPI

FastAPI is a Python web framework commonly used to expose APIs. For AI systems, it is often the thin application boundary around a Python orchestration or AI service: authentication, request validation, routing, dependency injection, async endpoints and API documentation can live here while agent logic lives in separate modules.

```
POST /api/v1/aira/investigate
{
```

```
"tenant_id": "MANYAVAR",
"question": "Why is order 10482 getting 403?"
}
```



19.1 FastAPI vs ASP.NET Core

For your background, think of FastAPI as a Python-side equivalent of the role ASP.NET Core Web API plays in the .NET ecosystem. You do not need to abandon .NET. Learning FastAPI gives you enough fluency to design and review Python AI services and communicate with AI-focused engineering teams.

20. Minimum Python knowledge required for AI

You do not need “full Python mastery” to become an AI-native solution architect. You need enough Python to read examples, write small services and tools, debug dependency issues, understand async execution, and modify AI/RAG/agent code safely. The official Python tutorial is designed to get programmers with general programming knowledge able to read and write Python modules and programs.

Python topic	Target level	Why you need it
Variables, strings, numbers, booleans	Comfortable	All AI code
Lists, tuples, sets, dictionaries	Comfortable	Messages, metadata, tool results
if / for / while	Comfortable	Control flow
Functions and parameters	Comfortable	Tools and service functions
Classes / objects	Basic	Framework models, state objects
Modules / imports / packages	Comfortable	LangChain ecosystem
Exceptions / try-except	Comfortable	API and tool failure handling
Type hints	Comfortable	Schemas, readability, validation
async / await	Basic to comfortable	FastAPI and concurrent I/O
Pydantic	Comfortable	Request / response schemas and structured data
Virtual environments / pip	Comfortable	Project setup
.env / environment variables	Comfortable	API keys and configuration
JSON / HTTP requests	Comfortable	API integrations / tools

Python topic	Target level	Why you need it
pytest basics	Basic	AI workflow tests and eval helpers

20.1 C# to Python mental mapping

C# / .NET	Python
class	class
interface	Protocol / abstract base class / duck typing
List<T>	list[T]
Dictionary<TKey,TValue>	dict[K,V]
Task / async	asyncio / async-await
NuGet	pip / uv / poetry-style package management
ASP.NET Core Web API	FastAPI
Data annotations / DTOs	Pydantic models
xUnit / NUnit	pytest

Minimum bar

You should be able to open a LangChain or LangGraph sample, understand 80-90% of the structure, change the model, add a tool, modify a schema, trace a failure and run the service. That is enough to progress into architecture without becoming a Python specialist.

21. Supporting skills you actually need

The named technologies sit on top of a small set of engineering foundations. These are part of your AI architecture toolkit.

Skill	Minimum depth	Application in AI systems
HTTP / REST	Strong	AI APIs, tools, MCP endpoints, external systems
JSON / schema design	Strong	Tool inputs, outputs, structured model responses
Pydantic / validation	Strong	Typed contracts between model and application
SQL	Strong	Business data retrieval and tools
Git	Strong	Prompts, agent code, eval datasets, infrastructure
Docker	Working knowledge	Package Python AI services consistently

Skill	Minimum depth	Application in AI systems
Azure	Architecture + hands-on	Identity, networking, containers, databases, monitoring
Concurrency	Conceptual + practical	Parallel tool calls and I/O performance
Testing	Strong engineering basics	Deterministic unit tests + AI evaluations
Observability	Strong	Trace, latency, errors, token usage, tool outcomes
Security	Strong	Prompt injection, data leakage, authorization, secrets
Data engineering	Conceptual	Ingestion, chunking, metadata, indexing, refresh

21.1 AI-specific concepts to add to your vocabulary

- Context window, input/output tokens, temperature / sampling concepts, reasoning vs non-reasoning modes.
- Structured output, JSON schema, tool calling and model-provider abstractions.
- Reranking, hybrid search, query rewriting, metadata filtering and citation grounding.
- Short-term state, long-term memory, summaries and conversation compaction.
- Guardrails, prompt injection, least privilege, human-in-the-loop and approval boundaries.
- Evaluation datasets, golden questions, retrieval metrics, answer correctness, groundedness, latency and cost.

22. Enterprise AI security and reliability

AI-native architecture changes the threat model because the model can influence which tools are called and what information is assembled into its context. The architecture must therefore treat model output as untrusted input until validated.

Risk	Architectural control
Prompt injection	Separate trusted instructions from untrusted content; tool permissions; output validation; content filtering.
Data leakage	Tenant-aware retrieval filters; field-level controls; PII detection; least privilege.
Unauthorized action	Application-side authorization; approval workflow; allowlisted tools; idempotency.
Hallucination / unsupported answer	RAG grounding; citations; confidence policy; abstain / escalate when evidence is weak.
Infinite loop / runaway cost	Max iterations, timeouts, budgets, cancellation, stop conditions.

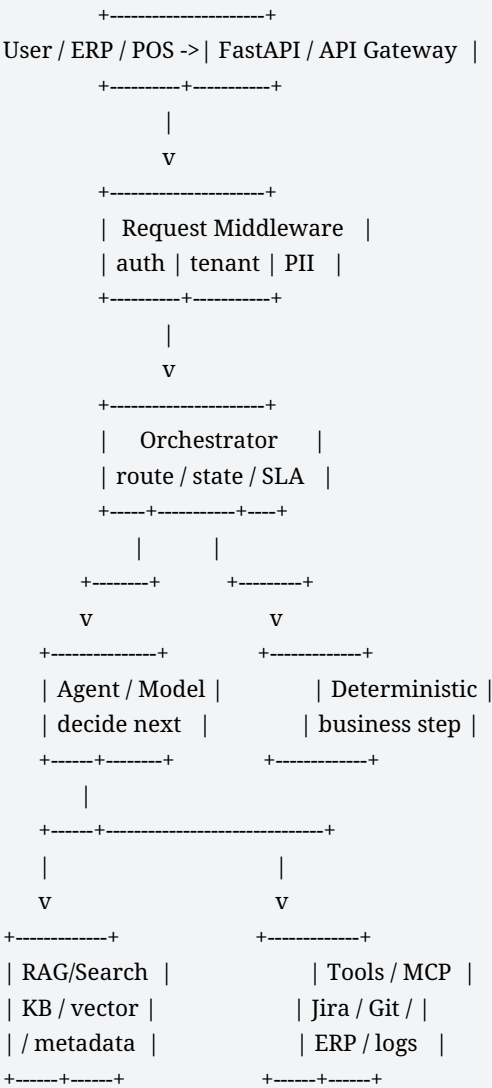
Risk	Architectural control
Tool failure	Retries for transient errors, circuit breakers, fallback paths, structured error handling.
Prompt / model regression	Version prompts and models; automated evaluations; trace comparison.

Trust boundary rule

The LLM can propose. Your deterministic application code should authorize, validate and execute.

23. AIRA - complete conceptual architecture

AIRA is the ideal project through which to learn these concepts because it naturally combines retrieval, agents, tools, APIs, state, orchestration, security and observability.



Step	System behavior	Main concept
	identifiers such as customer, API and request ID.	
3	Decide investigation plan: KB + recent incidents + logs.	Agent + orchestrator
4	Retrieve troubleshooting documents filtered to the tenant / product scope.	RAG
5	Search Jira for similar incidents.	Tool or MCP
6	Query logs for the request ID and status code.	Tool or MCP
7	Combine evidence and identify contradictions.	Agent reasoning / state
8	Run deterministic checks for known causes: token expiry, gateway policy, IIS binding, scope mismatch.	Program code
9	Generate structured diagnosis with evidence and confidence.	Structured output
10	If confidence is low or action is risky, request human review.	Guardrail / human-in-the-loop
11	Return answer with source references and trace ID.	Grounding + observability

24.1 What RAG contributes here

RAG provides knowledge such as “known 403 causes”, deployment runbooks and prior troubleshooting notes. It does not directly prove that customer ABC has a token-expiry problem today.

24.2 What tools contribute here

Tools retrieve current facts: current logs, current customer configuration, current Jira state, current service health. They move AIRA from static knowledge toward live investigation.

24.3 What the agent contributes here

The agent decides which evidence to collect and how to adapt when a result changes the next step. For example, if the logs show a valid token but a gateway policy rejection, the agent can shift the investigation toward gateway configuration rather than continue searching token-expiry articles.

24.4 What the orchestrator contributes here

The orchestrator imposes the process boundary: deadlines, maximum investigation steps, parallelization, approval checkpoints, retries and final response policy.

25. Choosing the right pattern

Requirement	Start with	Add when needed
Answer from a few static documents	Simple RAG	Hybrid search / reranking / better chunking
Current data from ERP / CRM	Tool calling	MCP when you need standardized reusable connectors
Predictable multi-step process	Deterministic workflow	LLM only at decision / summarization points
Open-ended investigation	Agent	LangGraph for explicit state and complex control flow
Enterprise cross-cutting controls	Middleware + application policy	More advanced guardrails / approval workflows
Need to debug AI behavior	Tracing	LangSmith + evaluation datasets
Python AI microservice	FastAPI	Containerization, auth, observability, async I/O

25.1 A practical escalation ladder

Level 0: normal API + business code

|

Level 1: LLM call

|

Level 2: LLM + structured output

|

Level 3: LLM + RAG

|

Level 4: LLM + tools

|

Level 5: Agent (model-driven tool loop)

|

Level 6: Agent + LangGraph orchestration/state

|

Level 7: Multi-agent / complex orchestration

Only move down the ladder when the use case requires it.

26. Common confusion map

When you hear...	Translate it to...
Agent	Model-driven decision loop + tools + state
Tool	Callable capability exposed to the model

When you hear...	Translate it to...
Function calling	Protocol/mechanism for requesting application-side tool execution
Orchestrator	Component controlling workflow topology, state and coordination
Workflow	Predefined sequence / graph of steps
Middleware	Interception layer for cross-cutting execution controls
MCP	Standardized way for AI applications to connect to external capabilities
RAG	Retrieve external evidence and give it to the model for grounding
Embedding	Semantic numeric representation
Vector DB	Storage/search system for vectors + metadata
Token	Model-processing unit of text
LangChain	LLM application/agent building blocks and abstractions
LangGraph	Stateful agent/workflow orchestration runtime
LangSmith	Tracing, monitoring and evaluation platform
FastAPI	Python HTTP API framework / service boundary

27. Study path and practical labs

Study these topics in dependency order. This prevents framework-first learning, where the syntax is memorized but the architecture remains unclear.

Phase	Study	Practical lab
1	LLM, tokens, prompts, structured output	Simple question-answer API
2	Tools / function calling	get_customer + get_order tools
3	Embeddings + vector search	Search 20 troubleshooting notes
4	RAG	AIRA knowledge assistant
5	LangChain agents + middleware	Agent chooses KB vs API tool
6	LangGraph	Explicit AIRA investigation state machine
7	MCP	Expose Jira / Git / ERP capabilities through MCP
8	LangSmith + evaluation	Trace all runs + 30 golden questions
9	FastAPI + Docker + Azure	Deploy AIRA service

Phase	Study	Practical lab
10	Security / cost / resilience	Tenant isolation + budgets + approval gates

27.1 Recommended minimum implementation milestones for AIRA

1. AIRA-01: User -> LLM -> answer.
2. AIRA-02: User -> LLM -> one deterministic tool -> answer.
3. AIRA-03: User -> RAG -> LLM -> grounded answer.
4. AIRA-04: User -> Agent -> RAG + tools -> answer.
5. AIRA-05: LangGraph -> state + deterministic checks + agent step + human approval.
6. AIRA-06: MCP -> Jira / GitHub / Confluence / ERP connectors.
7. AIRA-07: FastAPI + Docker + Azure deployment + LangSmith observability + evaluation.

28. Definition of an AI-native solution architect level

You do not need to be a model researcher. A strong AI-native solution architect should be able to design and defend an end-to-end solution, choose the minimum necessary AI pattern, identify trust boundaries, define data/retrieval strategy, create tool contracts, control orchestration, estimate cost/latency, and define evaluation and operational controls.

Can explain	Can design	Can implement enough to validate
LLM, tokens, embeddings, RAG, agents, MCP	Reference architecture and boundaries	Small vertical slices in Python
LangChain vs LangGraph vs LangSmith	Workflow / agent topology	Tools, retrievers, state graphs
Prompt vs code vs middleware	Security and approval model	Validation, retries, logging
Standard vs agentic RAG	Multi-tenant retrieval isolation	Embedding + vector search prototype
Model / tool / data responsibilities	Cost, latency and resilience strategy	FastAPI endpoint + container
Evaluation concepts	Production rollout / observability	Traces + evaluation set

29. One-page cheat sheet

Question	Answer to remember
What is AI?	A broad field; in this document we focus on application architecture around generative models.
What is an LLM?	A probabilistic language model that processes/generates token sequences.
Why tokens?	Models process text in tokens; tokens affect context, latency and usage.

Question	Answer to remember
Why embeddings?	To represent content numerically for semantic similarity and retrieval.
Why RAG?	To ground the model in external/private/current information.
Why tools?	To let the model access current data or perform application capabilities.
Why agents?	To let the model decide what tool/action to use next in an adaptive loop.
Why orchestrator?	To control the whole process: routing, state, timing, parallelism, failure, approvals.
Why middleware?	To apply cross-cutting controls and transformations around execution.
Why MCP?	To standardize connections between AI applications and external capabilities.
Why LangChain?	Reusable LLM/agent application abstractions and integrations.
Why LangGraph?	Explicit stateful orchestration for complex/long-running workflows and agents.
Why LangSmith?	Trace, debug, monitor and evaluate AI application behavior.
Why FastAPI?	A practical Python service/API boundary for AI applications.
How much Python?	Enough to read/write small AI services, tools and integrations; not full language mastery.

The mental picture

FastAPI is the front door. Middleware is the checkpoint. The orchestrator is the conductor. The agent is the adaptive investigator. RAG is the knowledge retrieval path. Tools/MCP are the hands and connections to external systems. LangChain is the toolkit. LangGraph is the workflow/state runtime. LangSmith is the flight recorder and evaluation desk. The LLM is the reasoning/language engine.

30. Source library and further reading

How this source list was selected: there is no single objective “most rated” global ranking for technical documentation. This list therefore prioritizes primary/official documentation, standards/specifications, maintainers’ references, and seminal research that is useful internationally and likely to remain valuable as implementations change.

Verification date: 2 October 2026. Use the linked official page as the source of truth when a framework API, service name, model name, or feature changes.

Source hierarchy: Primary = official vendor/project docs or protocol spec. Foundation = seminal research. Architecture = official cloud/reference architecture. Practice = maintainer guidance or engineering reference.

30.1 Core AI concepts

Concept	Authoritative source(s)	Why this source matters
AI / LLM foundation	OpenAI — Key concepts Attention Is All You Need — arXiv	• Primary API concepts: models, tokens, context, tool use, prompting and production concerns. • Seminal Transformer paper. Read for conceptual grounding, not for application coding.
Agent	LangChain — Agents Anthropic — Building Effective AI Agents Microsoft Foundry — Agent Service	• Current agent loop, model + tools + middleware, and agent harness concepts. • Excellent conceptual comparison of workflows, agents and orchestration patterns. • Enterprise agent lifecycle, tools, identity, security and deployment model.
Tools / function calling / structured output	OpenAI — Function calling OpenAI — Structured outputs	• Canonical tool/function-calling flow, schemas and tool execution patterns. • Schema-constrained outputs and when structured output differs from tool calling.
Orchestrator / workflow	LangGraph — Overview Microsoft — Agentic RAG architecture Anthropic — Building Effective AI Agents	• Stateful graph orchestration, durable execution and human-in-the-loop concepts. • Clear explanation of fixed pipelines versus agent-driven orchestration. • Architecture patterns for sequential, parallel, routing and agentic workflows.
Middleware	LangChain — Middleware Microsoft.Extensions.AI	• Current hooks and middleware controls for agent execution. • Shows middleware, telemetry, caching and provider-agnostic AI abstractions in .NET.
MCP - Model Context Protocol	MCP — Official introduction MCP — Official specification	• Primary conceptual overview: hosts, clients, servers and standardized access to tools/data. • Authoritative protocol requirements, capabilities, message model and security principles.
RAG	Microsoft — Design and develop a RAG solution Lewis et al. — RAG paper Microsoft — Agentic RAG	• Architecture, chunking, embeddings, retrieval, evaluation and design methodology. • Foundational research paper that introduced the modern RAG formulation. • When to move from fixed retrieval pipelines to agentic retrieval.

Concept	Authoritative source(s)	Why this source matters
Tokenization	Hugging Face — Tokenizer OpenAI — Key concepts OpenAI — tiktoken	• Concrete explanation of tokenization and token-to-text mapping. • Tokens, context and model input/output concepts in API applications. • Open-source tokenizer implementation used for OpenAI model token counting workflows.
Vectorization / embeddings	OpenAI — Embeddings Azure AI Search — Vector search overview	• Primary explanation of embeddings and semantic similarity use cases. • Vector retrieval architecture, indexing and search concepts in Azure.
Vector database / retrieval	Azure AI Search — Vector search overview pgvector — PostgreSQL vector similarity search	• Enterprise retrieval with vector and lexical search. • Practical reference for vector storage and similarity search in PostgreSQL.
Context window / context engineering	Anthropic — Effective context engineering for AI agents OpenAI — Key concepts	• Strong conceptual treatment of context selection, compression and agent performance. • Current token/context concepts and related API primitives.
Prompt engineering	OpenAI — Prompt engineering Anthropic — Building Effective AI Agents	• Primary guidance on instructions, message roles and prompt design. • Useful agent-level perspective on instructions and workflow design.
Memory	LangGraph — Memory Microsoft Foundry — Runtime components	• Short-term thread state and long-term application/user memory patterns. • Conversations, responses and agent state from an enterprise managed-agent perspective.
LangChain	LangChain — Overview LangChain — Agents	• Current framework role, abstractions and relationship with LangGraph and LangSmith. • Agent construction, tools and middleware in current LangChain.
LangGraph	LangGraph — Overview LangGraph — Memory	• Canonical graph/state/runtime concepts. • Practical state persistence and production memory concepts.
LangSmith	LangSmith — Observability LangSmith — Evaluation types LangSmith — Online evaluations	• Tracing and debugging for LLM and agent runs. • Offline and online evaluation patterns, including LLM-as-judge. • Production monitoring of live traces and quality.

30.2 Engineering, production and architecture skills

Concept	Authoritative source(s)	Why this source matters
FastAPI	FastAPI — Tutorial FastAPI — First Steps	• Official step-by-step tutorial and long-term reference. • Minimal API, OpenAPI and interactive documentation concepts.
Minimum Python	Python — Official tutorial Python — asyncio	• The right depth for an experienced programmer learning Python. • Essential for AI APIs, concurrent calls, streaming and tool orchestration.
Pydantic / typed contracts	Pydantic — Models and validation	• Typed validation and data contracts used heavily in modern Python APIs and AI systems.
Testing / evaluation discipline	pytest — Getting Started LangSmith — Evaluation types Microsoft — RAG design and evaluation	• Core Python testing reference for unit and integration tests. • AI-specific offline/online evaluation strategy. • End-to-end RAG evaluation considerations.
Observability	OpenTelemetry — What is OpenTelemetry? OpenTelemetry — Concepts	• Vendor-neutral standard for traces, metrics and logs. • Context propagation, signals, instrumentation and semantic conventions.
Security / guardrails	OWASP — Top 10 for LLM Applications 2026 MCP — Security and Trust & Safety Anthropic — Trustworthy agents in practice	• Current security risks for LLM applications; use as a threat-model checklist. • Authoritative MCP consent, data privacy, authorization and tool safety principles. • Practical guidance on human control, safety, transparency and privacy in agents.
Azure AI architecture / production	Microsoft — Baseline Foundry chat reference architecture Microsoft Foundry — Architecture Microsoft — Foundry Agent Service networking	• Production-oriented enterprise AI architecture with networking, identity and observability. • Foundry resource hierarchy, governance, data and security architecture. • Private networking and traffic flow for hosted/prompt agents.
.NET AI bridge	Microsoft.Extensions.AI	• Best current starting point for provider-agnostic AI abstractions in .NET.
Docker / containerization	Docker — What is Docker?	• Container fundamentals needed to package FastAPI/AIRA services.
Database / PostgreSQL for AI	pgvector — PostgreSQL vector similarity search Azure AI Search — Vector search overview	• Useful for combining relational ERP data and embeddings in PostgreSQL. • Alternative managed retrieval architecture for Azure workloads.

Concept	Authoritative source(s)	Why this source matters
HTTP / API contracts	FastAPI — First Steps / OpenAPI OpenAI — Function calling	• API routing, OpenAPI schemas and interactive contracts. • Typed tool contracts between an LLM and application code.

30.3 Recommended “source of truth” order

- For a concept: read the primary official / standards source first.
- For architecture trade-offs: read the official architecture reference next.
- For implementation: use the framework/library docs matching your chosen stack.
- For deep theory: use the seminal research papers selectively.
- When a source conflicts with a memory, blog, video or older tutorial, prefer the current primary documentation.

Architect rule: learn concepts from standards/primary docs; learn frameworks from their current docs; use blogs/videos only as secondary explanations.

31. Final architect reminder

When confused, draw the system

User -> API -> middleware -> orchestrator -> agent/workflow -> RAG/tools/MCP -> model -> validation -> response -> tracing/evaluation. Then ask which box owns the problem. This simple map prevents most terminology confusion.